

Chapter 1

Scripts and Live Scripts

So far we've typed all of our programs "at the prompt." If you're only writing a few lines, this isn't so bad. But what if you're writing a hundred? Retyping each line of code every time you want to change or test your program will be time-consuming and tedious. Luckily, you don't have to. In this chapter, we'll look at two ways to run many lines at once: *scripts* and *live scripts*.

But, before we get to that let's discuss the generic term - a program.

1.1 What is a program?¹

A **program** is a sequence of instructions that specifies how to perform a computation. The computation might be something mathematical, such as solving a system of equations or finding the roots of a polynomial, but it can also be a symbolic computation, such as searching and replacing text in a document or something graphical, like processing an image or playing a video.

The details look different in different languages, but a few basic instructions appear in just about every language:

input: Get data from the keyboard, a file, the network, or some other device.

output: Display data on the screen, save it in a file, send it over the network, etc.

math: Perform basic mathematical operations like addition and multiplication.

conditional execution: Check for certain conditions and run the appropriate code.

repetition: Perform some action repeatedly, usually with some variation.

Believe it or not, that's pretty much all there is to it. Every program you've ever used, no matter how complicated, is made up of instructions that look pretty much like these. So you can think of programming as the process of breaking a large, complex task into smaller

¹This section is from Allen Downey's book, Think Python, 2e Edition.

and smaller subtasks until the subtasks are simple enough to be performed with one of these basic instructions.

1.2 Scripts

A *script* is a file that contains a MATLAB program. When you run a script, MATLAB executes the commands in it, one after another, exactly as if you had typed them at the prompt. Scripts are also sometimes called *M-files* because they use the extension *.m*, short for MATLAB.

1.2.1 Your First Script

You can create scripts with any text editor or word processor, but the simplest way is to click the **New->Script** button in the upper-left corner of the MATLAB interface, which opens a text editor designed for MATLAB, simply called the "Editor".

To try it out, create a new script and enter the following code:

```
x = 5
```

Then press the **Save** button. A dialog window should appear where you can choose the filename and the folder where your script will go. Change the name to *myscript.m* and save it into any folder you like.

Now click the green **Run** button. You might get a message that says the script is not found in the current folder. If so, click the button that says Change Folder and it should run.

A handy way to discover the current folder in which the Command Window is operating is to issue the `pwd` command - for 'print working directory'. You can also issue the `ls` command to display a listing of the files and folders in the current working directory, for example

```
>> ls
bike_update.m      fibonacci1_live.pdf  mylivescript.mlx   myscript.m
fibonacci1_live.mlx fibonacci1.m         mylivescript.pdf   swap.m
```

Pro Tip: You will likely find you run scripts, or portions of a script, frequently while you are developing, testing and debugging. The keyboard shortcuts can speed up this iteration cycle. To run the entire script the **F5** key is equivalent to clicking the Run button and to typing the name of the script in the Command Window. To run just a portion of your script, highlight that section and press the **Ctrl+Enter** keys.

You can also run a script by typing its name in the Command Window and pressing **enter**. For example, if you enter `myscript`, MATLAB should execute your script and display the result:

```
>> myscript
x = 5
```

There are a few things to keep in mind when using scripts. First, you should not include the extension `.m` when you run a script. If you do, you'll get an error message like this:

```
>> myscript.m
Undefined variable "myscript" or class "myscript.m".
```

Second, when you name a new script, try to choose something meaningful and memorable. Don't choose a name that's already in use; if you do, you'll replace one of MATLAB's functions with your own (at least temporarily). You might not notice right away, but you might get some confusing behavior later.

Also, the name of the script cannot contain spaces. If you create a file named *my script.m*, MATLAB will complain when you try to run it:

```
>> my script
Undefined function or variable 'my'.
```

It can be hard to remember which folder a script is in. To keep things simple, for now, I suggest putting all of your scripts in one folder.

1.2.2 Why Scripts?

There are a few good reasons to use a script. When you're writing more than a couple of lines of code, it might take a few tries to get everything right. Putting your code in a script makes it easier to edit than typing it at the prompt. Likewise, if you're running a script repeatedly, it's much faster to type the name of the script than to retype the code! And you might be able to reuse a script from one project to the next, saving you considerable time across projects.

But the great power of scripts comes with great responsibility: you have to make sure that the code you are running is the code you think you are running. Whenever you start a new script, start with something simple, like `x = 5`, that produces a visible effect. Then run your script and confirm that you get what you expect. When you type the name of a script, MATLAB searches for the script in a *search path*, which is a sequence of folders. If it doesn't find the script in the first folder, it searches the second, and so on. If you have scripts with the same name in different folders, you could be looking at one version and running another.

If the code you are running is not the code you are looking at, you'll find debugging a frustrating exercise! So it's no surprise that this is the Third Theorem of Debugging:

Be sure that the code you are running is the code you think you are running.

Now that you've seen how to write a script, let's use one to do something a little more complicated.

1.2.3 Example: The Fibonacci Sequence

The *Fibonacci sequence*, denoted F , is a sequence of numbers where each number is the sum of the previous two. It's defined by the equations $F_1 = 1$, $F_2 = 1$, and, for $i > 2$,

$F_i = F_{i-1} + F_{i-2}$. The following expression computes the n th Fibonacci number:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right]$$

We can translate this expression into MATLAB, like this:

```
s5 = sqrt(5);
t1 = (1 + s5) / 2;
t2 = (1 - s5) / 2;
diff = t1^n - t2^n;
ans = diff / s5
```

I use temporary variables like `t1` and `t2` to make the code readable and the order of operations explicit. The first four lines have a semicolon at the end, so they don't display anything. The last line assigns the result to `ans`.

If we save this script in a file named *fibonacci1.m*, we can run it like this:

```
>> n = 10
>> fibonacci1
ans = 55.0000
```

Before calling this script, you have to assign a value to `n`. If `n` is not defined, you get an error:

```
>> clear n
>> fibonacci1
Undefined function or variable 'n'.

Error in fibonacci1 (line 9)
diff = t1^n - t2^n;
```

This script only works if there is a variable named `n` in the workspace; otherwise, you should get an error. MATLAB will tell you what line of the script the error is in and display the line.

Error messages can be helpful, but beware! In this example, the message says the error is in `fibonacci`, but the actual problem is that we have not assigned a value to `n`. And that brings us to the Fourth Theorem of Debugging:

Error messages tell you where the problem was discovered, not where it was caused.

Often you have to work backwards to find the line of code (or missing line) that caused the problem.

1.2.4 Floating-Point Numbers

In the previous section, the result we computed was 55.0000. Since the Fibonacci numbers are integers, you might have been surprised to see the zeros after the decimal point.

They are there because MATLAB performs calculations using floating-point numbers. With *floating-point numbers*, integers can be represented exactly, but most fractions cannot.

For example, if you compute the fraction $2/3$, the result is only approximate—the correct answer has an infinite number of 6s:

```
>> 2/3
ans = 0.6666
```

It's not as bad as this example makes it seem: MATLAB uses more digits than it shows by default. You can use the `format` command to change the output format:

```
>> format long
>> 2/3
ans = 0.6666666666666667
```

In this example, the first 14 digits are correct; the last one has been rounded off.

Large and small numbers are displayed in *scientific notation*. For example, if we use the built-in function `factorial` to compute $100!$, we get the following result:

```
>> factorial(100)
ans = 9.332621544394410e+157
```

The `e` in this notation is *not* the transcendental number known as e ; it's just an abbreviation for “exponent.” So this means that $100!$ is approximately 9.33×10^{157} . The exact solution is a 158-digit integer, but with double-precision floating-point numbers, we only know the first 16 digits.

You can enter numbers using the same notation.

```
>> speed_of_light = 3.0e8
speed_of_light = 300000000
```

Although the floating-point format can represent very large and small numbers, there are limits. The predefined variables `realmax` and `realmin` contain the largest and smallest numbers MATLAB can handle.

```
>> realmax
ans = 1.797693134862316e+308

>> realmin
ans = 2.225073858507201e-308
```

If the result of a computation is too big, MATLAB “rounds up” to infinity.

```
>> factorial(170)
```

```
ans = 7.257415615307994e+306
```

```
>> factorial(171)
ans = Inf
```

Division by zero also returns `Inf`.

```
>> 1/0
ans = Inf
```

For operations that are undefined, MATLAB returns `NaN`, which stands for “not a number.”

```
>> 0/0
ans = NaN
```

1.2.5 Comments

Short, simple programs are easy to read, but as they get bigger and more complex, it gets harder to figure out what they do and how. That’s what comments are for.

A *comment* is a line of text added to a program to explain how it works. It has no effect on the execution of the program; it is there for human readers. Good comments make programs more readable; bad comments are useless at best and misleading at worst.

To write a comment, you use the percent symbol (%) followed by the text of the comment.

```
>> speed_of_light = 3.0e8      % meters per second
speed_of_light = 300000000
```

The comment runs from the percent symbol to the end of the line. In this case it specifies the units of the value. In an ideal world, MATLAB would keep track of units and propagate them through the computation, but for now that burden falls on the programmer.

Avoid comments that are redundant with the code:

```
>> x = 5      % assign the value 5 to x
```

Good comments provide additional information that’s not in the code, like units in the example above, or the meaning of a variable:

```
>> p = 0      % position from the origin in meters
>> v = 100     % velocity in meters / second
>> a = -9.8    % acceleration of gravity in meters / second^2
```

If you use longer variable names, you might not need explanatory comments, but there’s a trade-off: longer variable names are clearer, but longer code can become harder to read. Also, if you’re translating from math that uses short variable names, it can be useful to make your program consistent with your math.

1.2.6 Documentation

Every script should provide *documentation*, which is a comment that explains what the script does and what its requirements are.

For example, I might put something like this at the beginning of *fibonacci1.m*:

```
% Computes a numerical approximation of the nth Fibonacci number.  
% Precondition: you must assign a value to n before running this script.  
% Postcondition: the result is stored in ans.
```

A *precondition* is something that must be true when the script starts in order for it to work correctly. A *postcondition* is something that will be true when the script completes.

If there is a comment at the beginning of a script, MATLAB assumes it's the documentation for the script. So if you type `help fibonacci1`, you get the contents of the comment (without the percent signs).

```
>> help fibonacci1  
Computes a numerical approximation of the nth Fibonacci number.  
Precondition: you must assign a value to n before running this script.  
Postcondition: the result is stored in ans.
```

That way, scripts that you write behave just like predefined scripts. You can even use the `doc` command to see your comment in the Help Window.

1.2.7 Assignment and Equality

For beginning programmers, a common source of confusion is assignment and the use of the equals sign.

In mathematics, the equals sign means that the two sides of the equation have the same value. In MATLAB, an assignment statement *looks* like a mathematical equality, but it's not.

One difference is that the sides of an assignment statement are not interchangeable. The right side can be any legal expression, but the left side has to be a variable, which is called the *target* of the assignment. So this is legal:

```
>> y = 1;  
>> x = y + 1  
x = 2
```

But this is not:

```
>> y + 1 = x  
y + 1 = x  
|  
Error: Incorrect use of '=' operator.  
To assign a value to a variable, use '='.  
To compare values for equality, use '=='.  
|
```

In this case the error message is not very helpful. The problem here is that the expression on the left side is not a valid target for an assignment.

Another difference between assignment and equality is that a mathematical equality is true (or false) for all eternity; an assignment statement is temporary. When you assign $\mathbf{x} = \mathbf{y} + 1$, you get the *current* value of $\mathbf{y}$. If $\mathbf{y}$ changes later, $\mathbf{x}$ does not get updated.

A third difference is that a mathematical equality is a statement that may or may not be true. In mathematics, $y = y + 1$ is a statement that happens to be false for all values of y . In MATLAB, $y = y + 1$ is a sensible and useful assignment statement. It reads the current value of $\mathbf{y}$, adds 1, and replaces the old value with the new value.

```
>> y = 1;  
>> y = y + 1  
y = 2
```

When you read MATLAB code, you might find it helpful to pronounce the equals sign as “gets” rather than “equals.” So $\mathbf{x} = \mathbf{y} + 1$ is pronounced “ $\mathbf{x}$ gets the value of $\mathbf{y}$ plus one.”

1.3 Live Scripts

A MATLAB *live script* is an interactive document that combines code, formatted text (e.g., equations), images and output in once place. In one sense it merges the code from a script with the execution of the command window. A live script allows you to run sections of your program independently, debugging as you go, while embedding the computation in a portable document that you can export and share.

1.3.1 Your First Live Script

While regular (non-live) scripts are stored as a text file with the *.m* extension, live scripts are stored as package of code, rich text and images. Luckily, MATLAB hides all this complexity from the user and what you see is a single document, stored as a *.mlx* file.

You can create a live script by clicking the **New->Live Script** button in the upper-left corner of the MATLAB interface. This opens the MATLAB “Live Editor”. Create a new live script and add the following:

1. Add a text block. This is rich text that can be formatted using the buttons on the top of the Live Editor or using markdown.
2. Add a code block with the line $\mathbf{x} = 5$.

Then press the **Save** button to save the file as *mylivescript.mlx*.

Now click the same green **Run** button (or hit F5) to run the script and see the result in the same document. You can still call the live script from the Command Window:

```
>> mylivescript  
x =
```


But that is probably a rare use-case.

In addition to saving the live script as an *.mlx* file, you can also export the document in a variety of formats. Here is what the PDF export might look like:

My first live script

```
x=5
```

```
x = 5
```

1.3.2 Example: The Fibonacci Sequence

Returning to our earlier example of a plain script, we can now combine the rich text of the description, including the expression for the Fibonacci number, into a live script with alternating text and code blocks. The resulting PDF would look something like this:

The Fibonacci Sequence

The *Fibonacci sequence*, denoted F , is a sequence of numbers where each number is the sum of the previous two.

The following expression computes the Fibonacci number:

$$F_n = \frac{1}{\sqrt{5}} \left[\left(\frac{1 + \sqrt{5}}{2} \right)^n - \left(\frac{1 - \sqrt{5}}{2} \right)^n \right]$$

Assign a value for n

```
n = 10;
```

The result t is stored in ``ans``.

```
s5 = sqrt(5);  
t1 = (1 + s5) / 2;  
t2 = (1 - s5) / 2;  
diff = t1^n - t2^n;  
ans = diff / s5
```

```
ans = 55.0000
```

1.4 Chapter Review

This chapter presented scripts and live scripts, suggesting reasons to use them. We computed elements of a Fibonacci sequence, but because we used floating-point numbers, the results were sometimes only approximate. And we saw how to add comments to a program to document what it does and explain how it works.

Here are some terms from this chapter you might want to remember.

An *M-file* is a file that contains a *script*, which is a sequence of MATLAB commands. An *MLX-file* is a file that contains a *live script*, which is a collection of MATLAB code, rich text and graphics. The *search path* is the list of folders where the interpreter looks for M-files.

A *precondition* is something that must be true when the script starts in order for it to work correctly; a *postcondition* is something that will be true when the script completes.

The *target* of an assignment statement is the variable on the left side.

Floating-point is a way to represent and store numbers in a computer. *Scientific notation* is a format for typing and displaying large and small numbers; for example, `3.0e8` represents 3.0×10^8 or 300,000,000.

A *comment* is part of a program that provides additional information about the program, but does not affect its execution.

In the next chapter, you'll learn how to write programs that perform repetitive tasks using loops.

1.5 Exercises

Before you go on, you might want to work on the following exercises.

Exercise 1.1. To test your understanding of assignment statements, write a few lines of code that swap the values of `x` and `y`. Put your code in a script called *swap.m* and test it.

If it works correctly, you should be able to run it like this:

```
>> x = 1, y = 2
x = 1
y = 2

>> swap

>> x, y
x = 2
y = 1
```

Exercise 1.2. Imagine that you are the operator of a bike-share system with two locations: Monterey and Pacific Grove.

You observe that every day 5 percent of the bikes in Monterey are dropped off in Pacific Grove, and 3 percent of the bikes in Pacific Grove get dropped off in Monterey. At the beginning of the month, there are 100 bikes at each location.

Write a script called *bike_update.m* that updates the number of bikes in each location from one day to the next. The precondition is that the variables `m` and `pg` contain the number of bikes in each location at the beginning of the day. The postcondition is that `m` and `pg` have been modified to reflect the net movement of bikes.

To test your program, initialize `m` and `pg` at the command prompt, then execute the script.

```
>> m = 100
>> pg = 100
>> bike_update
```

The script should display the updated values of `m` and `pg`, but not any intermediate variables.

Remember that bikes are countable things, so `m` and `pg` should always be integer values. You might want to use the `round` function to compute the number of bikes that move each day.

If you execute your script repeatedly, you can simulate the passage of time from day to day (you can repeat a command by pressing the `up` arrow and then `enter` - or by clicking the "Run" button repeatedly in the MATLAB editor).

What happens to the bikes? Do they all end up in one place? Does the system reach an equilibrium, does it oscillate, or does it do something else?

In the next chapter, we will see how to execute your script automatically and how to plot the values of `m` and `pg` over time.